Auditoria Técnica de Software

ELCOP · Gestão de TI

7.4/10

Nota geral do projeto

Stack identificada: C#, ASP.NET Core 10 MVC, EF Core 10, ASP.NET Identity, SQLite / SQL Server / PostgreSQL
Linhas de código (aprox.): 19800
Data da análise: 25/08/2026
Realizado por: Codex — Auditoria de Engenharia Sênior

Sumário Executivo

O ELCOP é uma aplicação MVC de gestão de ativos e demandas de TI, organizada em Domain, Application, Infrastructure e Web. A separação de dependências é coerente, os serviços de aplicação são testáveis e há sinais claros de cuidado com integridade de inventário, auditoria e concorrência. Para um repositório jovem, o estado de engenharia é bom e a base é apropriada para evolução controlada.

A segurança foi tratada de forma concreta: autenticação obrigatória por padrão, políticas por função, antiforgery, limite de tentativas, cookies seguros em produção, cabeçalhos de defesa e testes HTTP de autorização e mass assignment. Não foi confirmado segredo versionado, SQL dinâmico ou falha crítica de autorização na amostra auditada.

Os pontos que merecem ação antes de uma exposição mais ampla são principalmente de operação: o autocadastro permanece habilitado com domínios vazios, o health check de banco é público, migrations são executadas por cada instância no startup e as configurações de produção ainda precisam ser externalizadas e endurecidas. Há CI, lockfiles e testes, mas falta automatizar uma verificação de vulnerabilidades e ampliar cenários de falha/integração em infraestrutura real.

Painel de notas por dimensão

Dimensão	Nota
Arquitetura	**8.0/10**
Qualidade de Código	**8.0/10**
Débito Técnico	**6.5/10**
Segurança	**7.0/10**
Performance	**7.0/10**
Testes	**8.0/10**
Dependências	**7.0/10**

Arquitetura

A direção Web → Infrastructure → Application → Domain é observada nos projetos e na injeção de dependências. Application depende de contratos como IAppDbContext e IArmazenamentoArquivos, preservando testabilidade; Infrastructure concentra EF Core, Identity, Firebase e armazenamento. A lógica sensível de demandas fica no serviço/controller com regras explícitas e trilha de auditoria.

A maior decisão operacional ainda está embutida no processo de bootstrap: cada inicialização tenta aplicar migrations. Isso funciona bem em instalação única, mas deve sair do caminho de startup antes de múltiplas réplicas em nuvem.

Severidade	Achado	Local	Recomendação
MÉDIO	Migrations aplicadas por toda instância no startup DbInitializer chama Database.MigrateAsync durante a inicialização. Em deploy com mais de uma réplica, instâncias podem disputar a mesma migration, atrasar readiness ou falhar de forma intermitente.	src/Elcop.TI.Infrastructure/Persistence/DbInitializer.cs:34	Executar migrations em job/etapa única de deploy com lock operacional; deixar a aplicação apenas validar disponibilidade do banco.

Qualidade de Código

O código é legível, com nomes de domínio, nullable habilitado, interfaces nos serviços e comentários úteis nos pontos de segurança. A construção do formulário de demandas descarta explicitamente campos controlados pelo servidor, e a persistência preserva campos imutáveis de auditoria. Não foi observada duplicação estrutural ou tratamento silencioso de exceção na amostra central.

Severidade	Achado	Local	Recomendação
INFO	Detalhes de EF Core habilitados fora de Development EnableDetailedErrors é ativado para todos os provedores, sem condicionamento por ambiente. A aplicação trata exceções em produção, mas detalhes adicionais aumentam custo e podem enriquecer logs com contexto interno desnecessário.	src/Elcop.TI.Infrastructure/DependencyInjection.cs:43,66	Habilitar detalhes apenas em Development e manter logs de produção estruturados, sem dados sensíveis.

Débito Técnico

O histórico possui seis commits, um único contribuidor e cerca de duas semanas de atividade, caracterizando uma base recente com bus factor elevado. A documentação é rica e reduz parte desse risco; ainda assim, decisões de release, backup, recuperação e operação em múltiplas instâncias não estão codificadas no repositório.

Severidade	Achado	Local	Recomendação
MÉDIO	Risco de continuidade concentrado em um autor O histórico local registra um único contribuidor. Domínio, infraestrutura e segurança estão concentrados na mesma pessoa, o que torna revisão, suporte e transferência de conhecimento vulneráveis.	histórico Git do repositório	Instituir revisão por pares, registrar decisões de arquitetura e criar um runbook de deploy, backup e recuperação.

Segurança

A postura de segurança da aplicação é boa: FallbackPolicy protege rotas por padrão; ações sensíveis usam políticas; POSTs auditados usam antiforgery; login e registro recebem rate limit; o cookie é HttpOnly e é Secure em produção; e há CSP, HSTS, X-Frame-Options e nosniff. A validação de upload combina extensão, tamanho e assinatura, e o armazenamento local protege contra traversal.

Os riscos confirmados são de superfície operacional. O autocadastro padrão não aprova contas automaticamente, mas continua publicamente alcançável e sem domínio permitido; o endpoint de health do banco é anônimo, algo adequado apenas quando a borda de rede já o restringe.

Severidade	Achado	Local	Recomendação
MÉDIO	Autocadastro habilitado sem restrição de domínio A configuração padrão expõe o fluxo de registro e deixa DominiosPermitidos vazio. Embora AprovacaoAutomatica seja false, qualquer e-mail pode criar uma conta pendente, consumindo dados e exigindo triagem; a proteção por IP não impede abuso distribuído.	src/Elcop.TI.Web/appsettings.json:24-31; src/Elcop.TI.Web/Controllers/ContaController.cs:78-142	Em produção, desabilitar autocadastro se ele não for necessário ou restringir DominiosPermitidos; adicionar observabilidade e limites por e-mail/dispositivo conforme o perfil de exposição.
BAIXO	Health check de banco exposto sem autenticação O endpoint /health é explicitamente AllowAnonymous e executa checagem do DbContext. Ele revela disponibilidade do serviço a qualquer origem que alcance a aplicação.	src/Elcop.TI.Web/Program.cs:34-35, 174	Restringir /health à rede do balanceador/monitoramento, ou separar liveness público de readiness autenticado/interno.
BAIXO	AllowedHosts permissivo como padrão de configuração AllowedHosts está configurado como *. Em uma publicação pública, a configuração de produção deve listar os hosts reais para reduzir dependência de cabeçalhos Host não validados.	src/Elcop.TI.Web/appsettings.json:52	Definir AllowedHosts específico via configuração de produção e documentar o valor por ambiente.

Performance

As listagens auditadas são paginadas e usam AsNoTracking em leitura; o painel possui cache curto e o pipeline registra requisições acima de um segundo. Essas escolhas reduzem pressão inicial no banco e facilitam detectar degradação. Não foram medidos latência, carga ou plano de consultas, portanto não há alegação quantitativa de capacidade.

Severidade	Achado	Local	Recomendação
MÉDIO	Instrumentação ainda limitada para operação O projeto registra apenas requisições lentas e não contém integração de métricas, tracing distribuído ou alertas. Isso dificulta distinguir gargalo de banco, cache, upload ou dependência externa quando o uso crescer.	src/Elcop.TI.Web/Program.cs:122-141	Adicionar métricas e tracing compatíveis com o ambiente de hospedagem, além de métricas de banco, filas de upload e taxa de erros.

Testes

Há 67 declarações Fact/Theory em testes de serviços e integração WebApplicationFactory. A execução local de dotnet test concluiu com sucesso e reportou 69 testes aprovados na suíte de aplicação. Foram lidos testes HTTP que comprovam fallback de autenticação, políticas por perfil, health check e defesa contra mass assignment em demandas. O CI compila com warnings como erros, roda testes, verifica migrations e publica cobertura.

Severidade	Achado	Local	Recomendação
BAIXO	Cobertura é publicada, mas não há limiar de qualidade A pipeline coleta e publica Cobertura, porém não falha quando a cobertura cai. Isso permite regressões graduais sem sinalização de qualidade.	.github/workflows/ci.yml:43-56	Definir limiares graduais por camada e acompanhar cobertura de ramos para regras de negócio críticas, sem transformar a métrica em objetivo isolado.

Dependências

Os pacotes principais estão em versões explícitas de .NET/EF Core 10.0.11, e os projetos de teste possuem packages.lock.json; a CI restaura em locked mode. Isso favorece builds reproduzíveis. A tentativa de consulta automática de vulnerabilidades não pôde ser concluída neste sandbox porque o NuGet.Config do perfil do usuário não está acessível, portanto nenhuma afirmação de ausência de CVEs é feita.

Severidade	Achado	Local	Recomendação
MÉDIO	Verificação de vulnerabilidades não está automatizada na CI A pipeline restaura com lock e testa, mas não executa audit de dependências nem gera SBOM. A detecção de CVEs dependerá de ação manual ou de uma ferramenta externa não documentada.	.github/workflows/ci.yml:1-56	Adicionar auditoria de vulnerabilidades/SBOM na CI com política de exceções revisável e atualização periódica das dependências.

Roteiro de Recomendações Priorizadas

Ações ordenadas por prioridade, combinando impacto e urgência dos achados de cada dimensão.

#	Recomendação	Categoria	Esforço	Impacto
1	Endurecer configuração de produção	Segurança	Baixo	Alto — restringe autocadastro, hosts permitidos e visibilidade operacional
2	Separar migrations do startup da aplicação	Arquitetura	Médio	Alto — evita disputa e indisponibilidade em múltiplas réplicas
3	Automatizar auditoria de dependências e SBOM	Dependências	Baixo	Alto — reduz janela de exposição a CVEs
4	Restringir readiness do banco à rede interna	Segurança	Baixo	Médio — reduz exposição de estado operacional
5	Desabilitar detalhes de EF Core em produção	Qualidade de Código	Baixo	Médio — reduz dados diagnósticos desnecessários
6	Adicionar métricas, tracing e alertas	Performance	Médio	Médio — torna degradação investigável
7	Estabelecer revisão por pares e runbooks	Débito Técnico	Médio	Médio — reduz risco de continuidade
8	Definir gate de cobertura por camada	Testes	Baixo	Médio — evita erosão da suíte

Apêndice — Metadados do Repositório

Nome do projeto	ELCOP · Gestão de TI
Stack de linguagens	C#, ASP.NET Core 10 MVC, EF Core 10, ASP.NET Identity, SQLite / SQL Server / PostgreSQL
Linhas de código (aprox.)	19800
Data da análise	25/08/2026
Escopo	Leitura dirigida dos pontos de entrada, DI/persistência, autenticação/autorização, upload, serviços centrais, configuração, CI e testes. O diretório tmp e artefatos gerados foram excluídos da contagem de código.
Manifests encontrados	4 projetos de produção .csproj, 2 projetos de teste .csproj, 2 packages.lock.json
CI configurado	Sim — GitHub Actions: restore locked, build /warnaserror, validação de migrations, testes e publicação de cobertura.
Testes executados	dotnet test Elcop.TI.slnx --configuration Release --no-restore: sucesso; 69 testes aprovados reportados na suíte de aplicação.
Limitações	Build posterior não pôde terminar porque uma DLL de xUnit do projeto Web.Tests estava bloqueada por processo externo; consulta de vulnerabilidades NuGet foi bloqueada pelo acesso sandbox ao NuGet.Config do perfil. Não foram inferidas CVEs a partir disso.

Auditoria pontual baseada no estado local em 25/08/2026. Não substitui pentest, teste de carga ou revisão do ambiente de produção.